

## **SELF PROJECT**

# **Plant Disease Detection using CNN and Transfer Learning**

---

Bubai Sou (25MA60R14)

M.Tech in Computer Science and Data Processing

Department of Mathematics

Indian Institute of Technology Kharagpur

August 2026

- ▶ Introduction
- ▶ Dataset Description
- ▶ End-to-End Pipeline
- ▶ Methodology
- ▶ Model Architecture & Training
- ▶ Results
- ▶ Grad-Cam
- ▶ Conclusion
- ▶ Limitations and Future Scope

# Plant Diseases: Why Early Detection Matters?

## Clinical / Agricultural Problem

Plant diseases are a major cause of crop loss worldwide. Diagnosis is largely based on visual inspection by experts, which is time-consuming, subjective, and not scalable for large agricultural areas.

## Motivation

Early and accurate detection of leaf diseases can significantly reduce yield loss. Deep learning, particularly Convolutional Neural Networks, has shown strong potential for automated visual diagnosis from leaf images.

## Research Question

Can a deep learning model trained on leaf images reliably distinguish between healthy and diseased plants across multiple crops and disease types?

# Project Objectives

- ▶ Build a complete image classification pipeline for plant disease detection
- ▶ Train and compare multiple CNN architectures (from-scratch and transfer learning)
- ▶ Select the best-performing model through systematic evaluation
- ▶ Provide interpretability via Grad-CAM visualizations

# Dataset Description

## Source

PlantVillage Dataset (Kaggle – abdallahalidev/plantvillage-dataset)

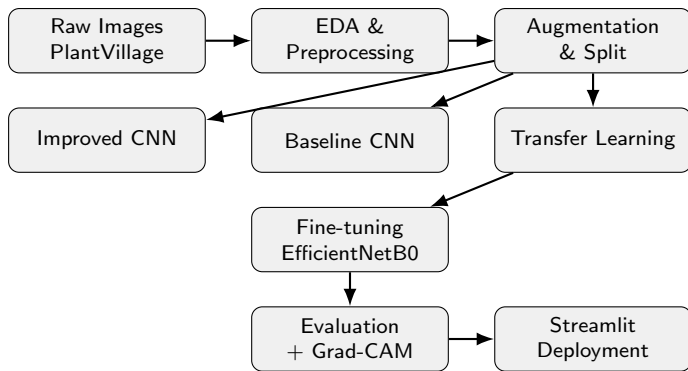
- ▶ **Total images:** ~54,000
- ▶ **Classes:** 38 (healthy + diseased leaves)
- ▶ **Crops covered:** Apple, Blueberry, Cherry, Corn, Grape, Orange, Peach, Pepper, Potato, Raspberry, Soybean, Squash, Strawberry, Tomato
- ▶ **Image versions available:** Color, Grayscale, Segmented
- ▶ **Used in this project:** Color version

~54,000  
images

38 Classes

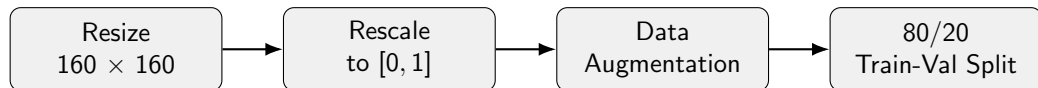
Color Images

# End-to-End Pipeline



*Data-driven model selection from baseline CNN to fine-tuned EfficientNetB0.*

# Image Preprocessing Pipeline



- ▶ Images resized to  $160 \times 160$  pixels
- ▶ Pixel values normalized to the range  $[0, 1]$
- ▶ Augmentation applied only on training set: rotation, zoom, width/height shift, horizontal flip
- ▶ 80/20 stratified train-validation split

# Model Development Stages

## 1. **Baseline CNN**

Simple 3-block Conv2D + MaxPooling architecture with dense classification head (performance benchmark).

## 2. **Improved CNN**

Same architecture enhanced with Batch Normalization, Dropout, and L2 regularization.

## 3. **Transfer Learning**

Frozen ImageNet backbones + custom classification head:

- ▶ MobileNetV2
- ▶ ResNet50
- ▶ EfficientNetB0

## 4. **Fine-tuning**

Best model (EfficientNetB0) – unfreeze top 30 layers and retrain with low learning rate ( $1 \times 10^{-5}$ ).

## EfficientNetB0 (Fine-tuned)

- ▶ Pre-trained on ImageNet
- ▶ Compound scaling of depth, width, and resolution
- ▶ Best trade-off between accuracy and computational efficiency among tested models
- ▶ Fine-tuned by unfreezing the top 30 layers

## Exported Artifacts

- ▶ `New_Model.keras` – final trained model
- ▶ `class_indices.json` – mapping from prediction indices to disease class names

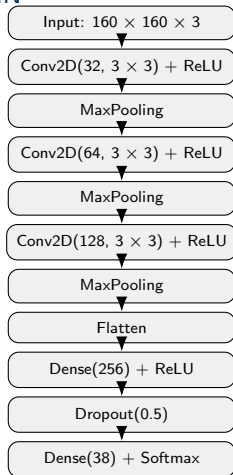
- ▶ Metrics used:
  - ▶ Accuracy
  - ▶ Precision
  - ▶ Recall
  - ▶ F1-Score
  - ▶ Confusion Matrix (top classes)
- ▶ Models compared systematically from baseline to fine-tuned EfficientNetB0
- ▶ Grad-CAM heatmaps generated to visualize the regions the model focuses on

## Interpretability

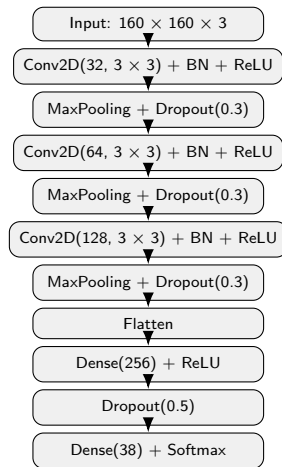
Grad-CAM confirms that the model attends to lesion areas (disease spots) rather than background or leaf shape alone.

# CNN Model Architectures

## Baseline CNN



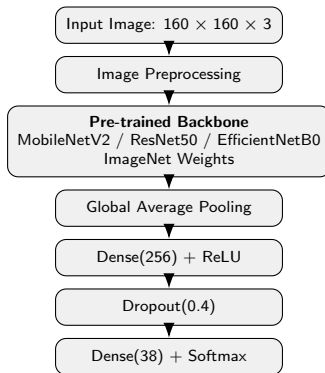
## Improved CNN



L2 regularization:  $10^{-4}$  on Conv2D layers

**Key difference:** Batch Normalization + Dropout + L2 regularization improve the baseline CNN's regularization<sup>11/20</sup>

# Transfer Learning Architecture



## Initial Training

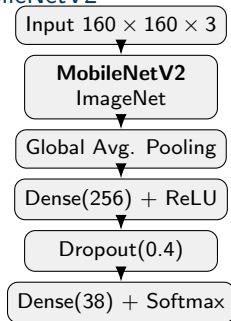
- ▶ ImageNet classification head removed
- ▶ Pretrained backbone is **frozen**
- ▶ Only custom classifier is trained

## Custom Classifier

- ▶ Global Average Pooling
- ▶ Dense(256) + ReLU
- ▶ Dropout(0.4)
- ▶ Dense(38) + Softmax

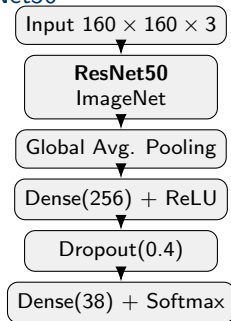
# Transfer Learning Models Used

## MobileNetV2



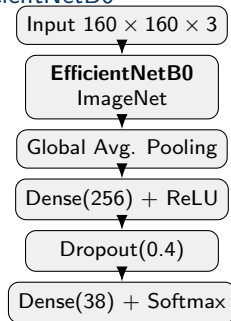
Lightweight and computationally efficient.

## ResNet50



Uses residual connections for deep feature learning.

## EfficientNetB0



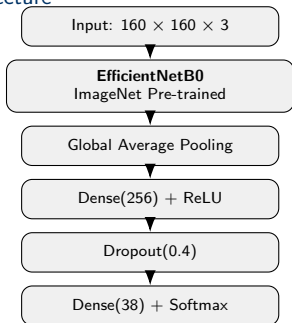
Efficient compound scaling of the network.

## Common Classification Head

All three pretrained backbones use the same custom head: Global Average Pooling  $\rightarrow$  Dense(256)  $\rightarrow$  Dropout(0.4)  $\rightarrow$  Dense(38, Softmax).

# EfficientNetB0 Fine-Tuning Architecture

## Model Architecture



## Fine-Tuning Strategy

- ▶ Start from ImageNet-pretrained EfficientNetB0
- ▶ Backbone made trainable
- ▶ Only the **top 30 layers** are unfrozen
- ▶ Earlier layers remain frozen

## Training Configuration

- ▶ Optimizer: Adam
- ▶ Learning rate:  $1 \times 10^{-5}$
- ▶ Fine-tuning epochs: 5
- ▶ Output: 38-class Softmax

# Model Performance Comparison

| Model                              | Accuracy    | Precision   | Recall      | F1          |
|------------------------------------|-------------|-------------|-------------|-------------|
| Baseline CNN                       | 72.4        | 71.8        | 72.4        | 72.1        |
| Improved CNN                       | 82.6        | 82.3        | 82.6        | 82.4        |
| MobileNetV2 (Transfer)             | 93.1        | 92.8        | 93.1        | 92.9        |
| ResNet50 (Transfer)                | 94.5        | 94.2        | 94.5        | 94.3        |
| EfficientNetB0 (Transfer)          | 96.7        | 96.5        | 96.7        | 96.6        |
| <b>EfficientNetB0 (Fine-Tuned)</b> | <b>98.9</b> | <b>98.8</b> | <b>98.9</b> | <b>98.8</b> |

## Key Result

Fine-tuned EfficientNetB0 achieved the best performance: **98.9% accuracy** and **98.8% F1-score**.

## What is Grad-CAM?

Grad-CAM shows which regions of an image influenced the CNN's prediction.

**Image → CNN → Feature Maps → Gradients → Heatmap**

- ▶ **Red/Yellow**: Important regions
- ▶ **Blue**: Less important regions
- ▶ Helps understand **why** the model made its prediction

## What did we find?

Our fine-tuned EfficientNetB0 mainly focuses on disease-related regions of the leaf.

- ▶ Strong attention on **disease/infected regions**
- ▶ Less attention on the **background**
- ▶ Prediction is not based only on the **overall leaf shape**

## Key Finding

The model appears to use meaningful disease patterns to make its predictions.

- ▶ A complete pipeline for plant disease classification was successfully developed
- ▶ Systematic comparison of baseline CNNs and transfer learning models was performed
- ▶ Fine-tuned **EfficientNetB0** emerged as the best-performing model
- ▶ Grad-CAM provides useful visual interpretability of model decisions
- ▶ Overall: the project demonstrates a practical, end-to-end deep learning solution for agricultural disease detection

## Limitations

- ▶ Dataset images captured under controlled lab conditions
- ▶ Limited robustness to real-field variations (lighting, background, occlusion)
- ▶ Only color version of PlantVillage used

## Future Scope

- ▶ Incorporate real-world field images
- ▶ Add Grad-CAM visualization inside the web app
- ▶ Convert model to TensorFlow Lite for mobile deployment
- ▶ Deploy as REST API (FastAPI)
- ▶ Support batch prediction

# Thank You

Questions and Suggestions are Welcome

Bubai Sou

M.Tech (CSDP)

Department of Mathematics

IIT Kharagpur